

ECM2433

Mock Paper 3

The C Family

Closed Book

Additional Materials: None

Permitted Materials: No calculators permitted

Guidelines:

The duration of this paper is 2hr 00.

Answer ALL questions.

Write all answers in the answer booklet provided.

Use only black pen or pencil.

Mobile and electronic devices are not allowed.

This is a mock paper in the style of the May paper. Possible answers are skeleton answers.

Answers and marking notes are at the end of this document.

Question 1

(a) Trace the conversions in the program. State the values printed for x, y, and u, and explain the integer division and unsigned result. (5 marks)

```
#include <stdio.h>

int main(void) {
    int a = 7;
    int b = 2;
    double x = a / b;
    double y = (double)a / b;
    unsigned int u = 0;

    u = u - 1;
    printf("x=%.1f y=%.1f u=%u\n", x, y, u);
}
```

(b) Complete copy_clean so it keeps only letters and digits, lowercases kept characters, writes a terminator, and returns 0 if the destination buffer is too small. (6 marks)

```
#include <ctype.h>
#include <stddef.h>

int copy_clean(char *dest, size_t dest_len, const char *src) {
    size_t write = 0;

    if (dest_len == 0) {
        return 0;
    }

    for (size_t read = 0; src[read] != '\0'; read++) {
        /* complete */
    }

    dest[write] = '\0';
    return 1;
}
```

(c) Fix the SQUARE macro so ordinary expressions are grouped correctly, then explain the remaining problem with SQUARE(i++). (4 marks)

```
#define SQUARE(X) X * X

int a = 2;
int b = 3;
int r = SQUARE(a + b);
```

(d) For files main.c, image.c, and image.h, write Makefile rules for main.o, image.o, and the executable viewer. Then state what rebuilds after image.h changes. (5 marks)

```
Project files:
main.c
image.c
image.h
```

```
Executable:
viewer
```

(e) Complete count_if and the call that counts even numbers in values using a function pointer predicate. (5 marks)

```
#include <stddef.h>

int is_even(int value) {
    return value % 2 == 0;
}

int count_if(const int *values, size_t len, int (*pred)(int)) {
    int count = 0;

    /* complete */

    return count;
}

int values[] = {3, 4, 8, 11, 12};
int evens = /* call count_if */;
```

(Total 25 marks)

Question 2

(a) For a C++ project containing main.cpp and sensor.cpp, write a minimal CMakeLists.txt and the two commands that configure into build/ and then build the executable. (4 marks)

```
Project files:
main.cpp
sensor.cpp
```

```
Executable:
sensor_app
```

(b) Complete the out-of-class constructor and const getter for Reading using a member initializer list. (5 marks)

```
class Reading {
    int id;
    double value;

public:
    Reading(int id, double value);
    double get_value() const;
};

/* definitions here */
```

(c) Complete the base class interface and the vector insertion so a TemperatureSensor is owned through a base-class smart pointer and read polymorphically. (6 marks)

```
#include <iostream>
#include <memory>
#include <vector>

class Sensor {
public:
    /* base interface */
};

class TemperatureSensor : public Sensor {
    int id;
    double celsius;

public:
    TemperatureSensor(int id, double celsius) : id(id), celsius(celsius) {}
    double read() const override { return celsius; }
};

std::vector<std::unique_ptr<Sensor>> sensors;
/* insert TemperatureSensor 7, 21.5 */

for (const auto& sensor : sensors) {
    std::cout << sensor->read() << "\n";
}
```

(d) The vector is sorted largest first. Complete the binary_search call correctly and explain why the same ordering rule is needed. (5 marks)

```
#include <algorithm>
#include <functional>
#include <vector>

std::vector<int> scores = {90, 70, 100, 80};

std::sort(scores.begin(), scores.end(), std::greater<int>{});

bool found = std::binary_search(/* complete */);
```

(e) Complete the function object so count_if counts readings whose value is above the stored limit. (5 marks)

```
#include <algorithm>
#include <vector>

struct Reading {
    int id;
    double value;
};

struct Above {
    double limit;

    /* function call operator */
};

std::vector<Reading> readings = {{1, 18.0}, {2, 22.5}, {3, 25.0}};
int hot = std::count_if(readings.begin(), readings.end(), Above{20.0});
```

(Total 25 marks)

Question 3

(a) Fix the code so name can still be printed after calculating its length. Use borrowing rather than cloning. (5 marks)

```
fn label_len(label: String) -> usize {
    label.len()
}

fn main() {
    let name = String::from("sensor");
    let n = label_len(name);
    println!("{name} {n}");
}
```

(b) Fix the derive line and the generic bounds so the function can compare two values and print the matching value with debug formatting. (5 marks)

```
struct Point {
    x: i32,
    y: i32,
}

fn show_if_same<T>(a: T, b: T) {
    if a == b {
        println!("{:?}", a);
    }
}
```

(c) Complete first_over_limit so it returns the first value greater than limit from a slice, without explicit indexing. (4 marks)

```
fn first_over_limit(values: &[i32], limit: i32) -> Option<i32> {
    /* complete */
}
```

(d) Complete parse_port for lines such as port=8080. It should reject a missing equals sign, the wrong key, and a bad integer using Result. (6 marks)

```
fn parse_port(line: &str) -> Result<u16, String> {
    /* complete */
}
```

(e) Given the library code, write one unit test that can call helper and one integration-test style assertion that uses only the public API. (5 marks)

```
fn helper(n: i32) -> i32 {
    n + 1
}

pub fn double_after_help(n: i32) -> i32 {
    helper(n) * 2
}
```

(Total 25 marks)

Question 4

(a) Complete `read_record` so it seeks to record index in a binary file and reads exactly one Record into `out`. (6 marks)

```
#include <stdio.h>

typedef struct {
    int id;
    double temperature;
} Record;

int read_record(FILE *fp, long index, Record *out) {
    /* complete */
}
```

(b) For the diamond-shaped inheritance code, explain why `b.id` is ambiguous, then show one qualified access and one inheritance change that removes the duplicate Base subobjects. (5 marks)

```
class Base {
public:
    int id;
};

class Left : public Base {};
class Right : public Base {};
class Bottom : public Left, public Right {};

Bottom b;
b.id = 3;
```

(c) Fix the thread spawn so the closure can safely use `text`. Then state what happens to `text` after the spawn. (5 marks)

```
use std::thread;

fn main() {
    let text = String::from("done");

    let handle = thread::spawn(|| {
        println!("{}", text);
    });

    handle.join().unwrap();
}
```

(d) Trace the reference count in the `Rc` code. State what the first print outputs and why `b` is still valid after `drop(a)`. (4 marks)

```
use std::rc::Rc;

let a = Rc::new(String::from("log"));
let b = Rc::clone(&a);

println!("{}", Rc::strong_count(&a));
drop(a);
println!("{}", b);
```

(e) A program prints a progress message without a newline before a long task. Add the call that forces it to appear immediately, and state where `setbuf` must be called if a custom `stdout` buffer is used. (5 marks)

```
#include <stdio.h>

int main(void) {
    char buffer[BUFSIZ];

    /* optional buffer setup */
    printf("Working...");
    /* force output before long task */

    return 0;
}
```

(Total 25 marks)

Answers and marking notes

These are skeleton answers. Equivalent correct code and reasoning should receive credit.

Question 1

(a) Question (5 marks): Trace the conversions in the program. State the values printed for x, y, and u, and explain the integer division and unsigned result.

Answer: x is 3.0 because a / b is integer division before being converted to double. y is 3.5 because casting a to double makes the division floating-point. u wraps to UINT_MAX because unsigned arithmetic is modulo the range of the unsigned type; on a common 32-bit unsigned int this prints 4294967295.

(b) Question (6 marks): Complete copy_clean so it keeps only letters and digits, lowercases kept characters, writes a terminator, and returns 0 if the destination buffer is too small.

Answer: Use separate read and write indexes. Only kept characters advance write. Leave one byte for the terminator, and use the ctype functions on an unsigned char value.

```
for (size_t read = 0; src[read] != '\0'; read++) {
    unsigned char ch = (unsigned char)src[read];

    if (isalnum(ch)) {
        if (write + 1 >= dest_len) {
            dest[write] = '\0';
            return 0;
        }
        dest[write++] = (char)tolower(ch);
    }
}
```

(c) Question (4 marks): Fix the SQUARE macro so ordinary expressions are grouped correctly, then explain the remaining problem with SQUARE(i++).

Answer: The safer macro is fully parenthesised, but it still evaluates its argument twice. SQUARE(i++) can increment i twice, so a function or inline function is safer for arguments with side effects.

```
#define SQUARE(X) ((X) * (X))
```

(d) Question (5 marks): For files main.c, image.c, and image.h, write Makefile rules for main.o, image.o, and the executable viewer. Then state what rebuilds after image.h changes.

Answer: Both object files depend on image.h, so if image.h changes, main.o and image.o are rebuilt and then viewer is relinked.

```
CC = gcc
CFLAGS = -std=c99 -Wall -Wextra

viewer: main.o image.o
<TAB>$(CC) $(CFLAGS) main.o image.o -o viewer

main.o: main.c image.h
<TAB>$(CC) $(CFLAGS) -c main.c -o main.o

image.o: image.c image.h
<TAB>$(CC) $(CFLAGS) -c image.c -o image.o
```

(e) Question (5 marks): Complete count_if and the call that counts even numbers in values using a function pointer predicate.

Answer: The function pointer pred stores the address of a function that takes an int and returns an int. count_if calls that predicate for each element and increments the counter when it returns non-zero.

```
for (size_t i = 0; i < len; i++) {
    if (pred(values[i])) {
        count++;
    }
}

int evens = count_if(values, sizeof values / sizeof values[0], is_even);
```

Question 2

(a) Question (4 marks): For a C++ project containing main.cpp and sensor.cpp, write a minimal CMakeLists.txt and the two commands that configure into build/ and then build the executable.

Answer: CMake configures the build system into the build directory, then cmake --build drives the chosen native build tool to compile and link the executable.

```
cmake_minimum_required(VERSION 3.10)
project(sensor_app)
```

```
add_executable(sensor_app main.cpp sensor.cpp)

cmake -S . -B build
cmake --build build
```

(b) Question (5 marks): Complete the out-of-class constructor and const getter for Reading using a member initializer list.

Answer: The initializer list initializes the private data members before the constructor body. The getter is const because reading the value should not modify the object.

```
Reading::Reading(int id, double value) : id(id), value(value) {}

double Reading::get_value() const {
    return value;
}
```

(c) Question (6 marks): Complete the base class interface and the vector insertion so a TemperatureSensor is owned through a base-class smart pointer and read polymorphically.

Answer: The base class needs a virtual destructor because deletion happens through a base pointer type. read must be virtual so the TemperatureSensor implementation runs through the base smart pointer.

```
class Sensor {
public:
    virtual ~Sensor() = default;
    virtual double read() const = 0;
};

sensors.push_back(std::make_unique<TemperatureSensor>(7, 21.5));
```

(d) Question (5 marks): The vector is sorted largest first. Complete the binary_search call correctly and explain why the same ordering rule is needed.

Answer: binary_search assumes the range is sorted according to the same comparison it receives. Since the vector was sorted with greater<int>, the search must use greater<int> too.

```
bool found = std::binary_search(
    scores.begin(),
    scores.end(),
    80,
    std::greater<int>{}
);
```

(e) Question (5 marks): Complete the function object so count_if counts readings whose value is above the stored limit.

Answer: A function object is an object that can be called like a function because it overloads operator(). count_if calls it once for each element.

```
bool operator()(const Reading& reading) const {
    return reading.value > limit;
}
```

Question 3

(a) Question (5 marks): Fix the code so name can still be printed after calculating its length. Use borrowing rather than cloning.

Answer: Taking String by value moves ownership into label_len. Borrow text instead, so the caller keeps ownership and can still print name.

```
fn label_len(label: &str) -> usize {
    label.len()
}

fn main() {
    let name = String::from("sensor");
    let n = label_len(&name);
    println!("{name} {n}");
}
```

(b) Question (5 marks): Fix the derive line and the generic bounds so the function can compare two values and print the matching value with debug formatting.

Answer: The == operator needs PartialEq. Debug formatting with {:?} needs Debug. The custom struct must derive the same traits if it is used with this function.

```
#[derive(Debug, PartialEq)]
struct Point {
    x: i32,
    y: i32,
}

fn show_if_same<T>(a: T, b: T)
where
    T: PartialEq + std::fmt::Debug,
{
```

```

    if a == b {
        println!("{}", a);
    }
}

```

(c) Question (4 marks): Complete `first_over_limit` so it returns the first value greater than `limit` from a slice, without explicit indexing.

Answer: `find` returns `Option` because there may be no matching element. `copied` turns borrowed `i32` values from `iter` into owned `i32` values.

```

fn first_over_limit(values: &[i32], limit: i32) -> Option<i32> {
    values.iter().copied().find(|value| *value > limit)
}

```

(d) Question (6 marks): Complete `parse_port` for lines such as `port=8080`. It should reject a missing equals sign, the wrong key, and a bad integer using `Result`.

Answer: `split_once` gives an `Option`, so convert the missing case into an `Err`. Parsing the value gives a `Result`, so map the parse error into the function's `String` error type.

```

fn parse_port(line: &str) -> Result<u16, String> {
    let (key, value) = line
        .split_once('=')
        .ok_or_else(|| "missing equals sign".to_string());

    if key.trim() != "port" {
        return Err("expected port key".to_string());
    }

    value
        .trim()
        .parse::<u16>()
        .map_err(|_| "bad port".to_string())
}

```

(e) Question (5 marks): Given the library code, write one unit test that can call helper and one integration-test style assertion that uses only the public API.

Answer: Unit tests inside the same module can use `super` to reach private items. Integration tests in `tests/` use the crate like outside code, so they can only use public API.

```

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn helper_adds_one() {
        assert_eq!(helper(4), 5);
    }
}

// In tests/basic.rs, using the real crate name:
use my_crate::double_after_help;

#[test]
fn public_api_doubles_after_help() {
    assert_eq!(double_after_help(4), 10);
}

```

Question 4

(a) Question (6 marks): Complete `read_record` so it seeks to record index in a binary file and reads exactly one `Record` into out.

Answer: The byte offset is index multiplied by the size of one `Record`. `fseek` positions the stream, and `fread` must read exactly one object.

```

int read_record(FILE *fp, long index, Record *out) {
    long offset = index * (long)sizeof *out;

    if (fseek(fp, offset, SEEK_SET) != 0) {
        return 0;
    }

    return fread(out, sizeof *out, 1, fp) == 1;
}

```

(b) Question (5 marks): For the diamond-shaped inheritance code, explain why `b.id` is ambiguous, then show one qualified access and one inheritance change that removes the duplicate `Base` subobjects.

Answer: `Bottom` contains two `Base` subobjects: one through `Left` and one through `Right`. `b.id` is ambiguous at compile time. Qualified access chooses one path; virtual inheritance shares one `Base` subobject.

```

b.Left::id = 3;

```



```
class Left : virtual public Base {};  
class Right : virtual public Base {};  
class Bottom : public Left, public Right {};
```

(c) Question (5 marks): Fix the thread spawn so the closure can safely use text. Then state what happens to text after the spawn.

Answer: thread::spawn needs a closure that can outlive the current stack frame. move transfers ownership of text into the closure. After that, main cannot use text unless it cloned it before the move.

```
let handle = thread::spawn(move || {  
    println!("{text}");  
});
```

(d) Question (4 marks): Trace the reference count in the Rc code. State what the first print outputs and why b is still valid after drop(a).

Answer: The first print outputs 2 because a and b are both strong owners of the same allocation. drop(a) removes one owner, but b still owns the String, so printing b is valid. The object is destroyed when the final strong owner is dropped.

(e) Question (5 marks): A program prints a progress message without a newline before a long task. Add the call that forces it to appear immediately, and state where setbuf must be called if a custom stdout buffer is used.

Answer: setbuf must be called before any output on that stream. fflush(stdout) forces buffered stdout output to be written immediately.

```
setbuf(stdout, buffer);  
printf("Working...");  
fflush(stdout);
```